

ANAÁLISIS 3 CLAUDE DE ARC REASONER

Aquí va un roadmap de 12 semanas pensado para eso.

Principio guía: código feo en semana 2, no en semana 8

La arquitectura V3 que ya tienen es buena — el error sería tratar de implementarla completa antes de tocar un solo dato real de ARC. En vez de eso, cada sprint agrega una capa vertical delgada que atraviesa todo el pipeline (percepción → razonamiento → verificación), aunque cada capa sea primitiva.

Semanas 1-2 — Esqueleto end-to-end (el "hola mundo" del reasoner)

Estructuras mínimas: Grilla, Posición, Pieza, Estado (sin optimizar, sin abstracciones bonitas).

Segmentación más simple posible: componentes conexos por color (4-conectividad).

Motor de comparación con una sola categoría de diferencia: traslación de objeto.

Generador de hipótesis: si detecta traslación uniforme en todos los objetos → aplica la misma traslación.

Meta de la semana 2: correr sobre 5 tareas de ARC-1 elegidas a mano (las más triviales) y resolver al menos 2-3.

Esto no prueba nada sobre ARC en general, prueba que el pipeline completo corre sin bloquearse.

Semanas 3-4 — Ampliar el catálogo de transformaciones

Sumar: cambio de color, escalado, rotación/reflexión simple, eliminación/creación de objetos.

Motor de comparación ahora produce lista de diferencias con estas categorías.

Generador de hipótesis sigue siendo basado en reglas fijas (if/else), no búsqueda todavía.

Meta: correr sobre el training set completo de ARC-1 (400 tareas) y medir accuracy real. Aunque sea 3-5%, ya es una métrica objetiva y reproducible.

Semanas 5-6 — De reglas fijas a búsqueda de hipótesis

Aquí es donde el proyecto se pone interesante de verdad: convertir el generador de hipótesis en una búsqueda sobre un espacio de transformaciones compuestas (DSL pequeño), con poda por consistencia entre los ejemplos de entrenamiento de cada tarea.

No hace falta que sea elegante — un beam search simple con 20-30 primitivas ya da señal.

Meta: duplicar o triplicar el accuracy de la semana 4. Documentar qué tipo de tareas siguen fallando (esto define el roadmap de percepción).

Semanas 7-8 — Reforzar percepción según los fallos reales

Con datos reales de qué falla (semana 6), decidir con evidencia qué mejorar: ¿objetos con huecos? ¿simetría? ¿relaciones topológicas (adentro/afuera, tocando)? ¿grids con múltiples subgrillas?

Esto evita diseñar percepción "porque suena completa" — se construye lo que el benchmark realmente exige.

Meta: subir accuracy de nuevo, y tener el Grafo de Relaciones funcionando para al menos relaciones espaciales básicas.

Semanas 9-10 — Verificación robusta + manejo de incertidumbre

Implementar el módulo de Verificación como filtro real: aplicar cada hipótesis candidata a los ejemplos de entrenamiento, descartar las que no reproducen el resultado exacto.

Sistema de ranking cuando hay múltiples hipótesis válidas (ARC permite 2 intentos por tarea — esto importa para la métrica oficial).

Meta: correr con la métrica real de ARC Prize (top-2 attempts) y tener un número comparable a benchmarks públicos.

Semanas 11-12 — Empaquetado del MVP y la cita

Congelar código, limpiar el pipeline lo mínimo necesario para que sea demostrable (no hace falta que sea production-grade).

Preparar: (a) demo en vivo resolviendo tareas paso a paso mostrando el razonamiento explícito — esto es tu diferenciador frente a un LLM caja negra, (b) número de accuracy en training set, (c) 2-3 ejemplos donde el sistema "explica" su respuesta (percepción → diferencias → hipótesis → verificación), porque eso es lo que un inversor no va a ver en un modelo neuronal puro.

Meta: MVP demostrable con narrativa clara de por qué el enfoque simbólico/explicable importa, no solo el número.

Sobre "no sé qué aplicar aparte de ARC"

Esto es importante para la cita — un inversor va a preguntar "¿y esto para qué sirve más allá del benchmark?". La ventaja real de tu arquitectura no es ARC en sí, es que es explicable e inspeccionable por diseño (cada paso produce un objeto intermedio legible: diferencias, hipótesis, verificación). Eso abre mercados donde "caja negra" es inaceptable:

Control de calidad visual / manufactura: detectar qué cambió entre una pieza esperada y una producida, y explicar el defecto en términos estructurados (no solo "hay una anomalía").

Automatización de diseño gráfico/UI: inferir reglas de transformación entre versiones de un layout (ej. "todos los botones se movieron 20px y cambiaron de color") para generar variantes automáticamente.

Verificación de diagramas técnicos / CAD: comparar dos versiones de un plano y explicar la diferencia en lenguaje estructurado.

Tutoría/educación: sistemas que no solo dan la respuesta sino que explican la transformación paso a paso — útil en literalidad para enseñar razonamiento espacial a niños o en robótica educativa.

No necesitas resolver esto ahora, pero sí vale la pena mencionar en la cita una aplicación concreta con una frase, no una lista — los inversores reaccionan mejor a "esto es un motor de razonamiento visual explicable, ARC es nuestro benchmark de validación, la primera vertical comercial que exploraremos es X" que a un menú de posibilidades.

¿Quieres que arme esto como documento (con tabla de sprints, métricas y entregables) para llevar a la reunión, o prefieres quedarte con esta versión conversacional por ahora?

Buena pregunta — y el criterio correcto para decidir no es "cuál mercado es más grande" sino "cuál vertical necesita exactamente lo que tu arquitectura ya hace bien versus lo que todavía no hace bien". Vamos por partes.

Lo que tu arquitectura realmente sabe hacer (hoy, conceptualmente)

Detectar diferencias estructuradas entre dos estados espaciales/relacionales y explicar la transformación. Eso es fundamentalmente ****comparación de configuraciones espaciales con salida auditable****. No es razonamiento médico, no es optimización logística — es geometría + relaciones + explicación.

Con eso en mente:

Logística

****A favor:****

- Encaja casi literalmente con tu modelo conceptual. Un almacén **es** una grilla. Un pallet, un contenedor, un SKU **es** una pieza. "¿Dónde estaba el inventario ayer y dónde está hoy, y qué transformación lo explica?" es exactamente tu Motor de Comparación de Estados sin metáfora — es el problema real, sin traducción.
- Casos concretos donde esto vende solo:
 - Auditoría de inventario en almacenes (detectar movimientos no registrados, comparar layout esperado vs. real desde cámaras/sensores).
 - Verificación de picking/packing (¿el pedido armado coincide estructuralmente con el pedido solicitado? ¿qué cambió — faltante, sustitución, posición incorrecta?).
 - Optimización de layout (comparar configuraciones de estantería antes/después de un rediseño y verificar que la transformación aplicada es consistente en todo el almacén).
- Los datos de entrenamiento/validación son mucho más fáciles de conseguir y de simular que en salud (no hay barrera regulatoria para experimentar).
- El ciclo de venta es más corto: un gerente de operaciones puede aprobar un piloto sin pasar por comités de ética o compliance HIPAA.

****En contra:****

- Es un mercado con jugadores establecidos (sistemas WMS, visión por computadora industrial) que ya resuelven partes de esto, aunque casi todos como caja negra estadística — tu ángulo de "explicabilidad estructurada" sigue siendo diferenciador, pero tendrías que posicionarte contra soluciones ya vendidas, no contra un vacío.

Salud

****A favor:****

- "Auditable" e "interpretable" tienen mucho más peso de marketing/regulatorio en salud — es literalmente lo que reguladores como la FDA piden para aprobar software como dispositivo médico (SaMD): trazabilidad de cada decisión.
- Casos donde la comparación estructurada de estados brilla: comparar dos imágenes médicas en el tiempo (¿qué cambió en un tumor, una fractura, una radiografía entre dos fechas, expresado como transformación explicable en vez de un score de red neuronal?), o auditoría de expedientes/protocolos.

****En contra — y esto es serio:****

- El costo de entrada es enormemente mayor: ciclos regulatorios (FDA/CE), necesitas datos anotados por especialistas (caros, escasos, protegidos por HIPAA/GDPR), y el margen de error tolerado es prácticamente cero — un "falso negativo" en salud no es un pallet mal ubicado, es un paciente.
- Con un equipo pequeño y un MVP de 3 meses, es prácticamente imposible mostrar algo demostrable y creíble en salud sin datos reales de dominio médico, que hoy no tienes.

- Un inversor serio en salud va a preguntar por validación clínica, no por accuracy en ARC — y ahí no tienes nada que mostrar todavía.

Mi recomendación

****Logística para el MVP y la cita, salud como visión a largo plazo (mencionada, no prometida).****

La razón no es que salud sea peor negocio — es mejor negocio a largo plazo, de hecho, precisamente porque la barrera regulatoria protege a quien ya está adentro. Pero con 3 meses de plazo y un motor que hoy resuelve tareas ARC, necesitas la vertical con ****menor distancia entre lo que tu sistema ya hace y lo que el mercado necesita ver****. Esa es logística, sin ambigüedad.

Para la cita, yo diría algo como: **"Estamos validando el motor sobre ARC como benchmark de razonamiento espacial explicable. La primera vertical comercial es auditoría de inventario y detección de discrepancias en logística, donde la explicabilidad no es un nice-to-have sino un requisito operativo. A mediano plazo, la misma arquitectura de comparación estructurada de estados tiene aplicación directa en imagenología médica comparativa — pero esa vertical la abordamos después de validar el approach en un dominio de menor fricción regulatoria."**

Eso te da: una historia coherente, un mercado donde puedes mostrar tracción real en 3 meses, y una narrativa de expansión a salud que suena a estrategia, no a promesa vacía.